execution and return before testing the outcome, using an `assert` statement. The first `it()` demonstrates a positive test case, that of a successful vote by a registered account holder, and the second `it()` demonstrates a negative test case in which a voter account votes for a nonexistent proposal number. Hence, the latter should revert. Now study the code to understand the concepts of testing so that you can write your test script for your smart contract:

```
describe('Voting', function() {
    beforeEach('Setup contract for each voting test', async function () {
      // register two accounts
      await ballot.register(accounts[1], { from: accounts[0]})
      await ballot.register(accounts[2], { from: accounts[0]})
    });

    it('Success on vote.', async function () {
      //Registration -> Vote
      await ballot.changeState(2)
      let result = await ballot.vote(1, { from: accounts[1]})
      assert.equal(result.receipt.status, success)
      result = await ballot.vote(1, { from: accounts[2]})
      assert.equal(result.receipt.status, success)
    });

    it('Failure on voting for invalid candidate.', async function () {
      //Registration -> Vote
      await ballot.changeState(2)

      //number of proposals is 3: must fail when trying to vote for 10.
      await truffleAssert.reverts(
        ballot.vote(10, { from: accounts[1]}), wrongProposalError
      )
    });
```

10.4 Recap writing of test script

Now that you've seen two examples, let's recap the structure of the test scripts and an approach to coding one. The general structure of the test script is shown in figure